



M256 Unit 2

UNDERGRADUATE COMPUTING

Software development with Java



Requirements
specification

Unit **2**

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)870 333 4340, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; email ouwenq@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007.

Copyright © 2007 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 1563 7

The background of the page is a teal color. On the left side, there is a vertical strip featuring abstract, glossy spheres of varying sizes that appear to be floating above a body of water, with their reflections visible below. The spheres have a bright highlight on their upper left, giving them a three-dimensional appearance.

CONTENTS

1	Introduction	5
1.1	The aims of this unit	5
1.2	Studying this unit	6
2	Eliciting and analysing requirements	7
2.1	The role of analysts	7
2.2	What makes a good analyst?	9
2.3	Techniques used to elicit requirements	11
2.4	Asking questions	11
2.5	Analysis and specification of use cases	13
2.6	Requirements for user interfaces	16
3	The requirements document	17
3.1	A template for a requirements document	17
3.2	The main sections of a requirements document	18
3.3	Constraints	20
3.4	The requirements document as a contract	23
4	The Hospital System	24
4.1	The starting point	24
4.2	Asking questions	24
4.3	The requirements document	29
5	The Shopping System	32
5.1	A brief description of the Shopping System	32
5.2	The requirements document	35
6	Summary	39
	Glossary	40
	References	41
	Index	42

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

Unit 1 introduced you to the phases of software development that you will study in this course. In this unit you will learn about the first of these phases, requirements specification.

Requirements specification as defined in this course consists of two activities:

- 1 *eliciting* the requirements of a software system from the client and future users of a system;
- 2 *analysing* these requirements to produce a specification of what the system must do.

As such, the requirements specification phase plays a vital role in software development, because it is the stage at which the tasks to be performed by a new system are identified and agreed to (often contractually) by the software developers and the client commissioning the new system. It is easy to see that any mistakes in specifying these tasks will have serious ramifications for the whole project.

Unit 2 will not make you an expert in requirements specification, but you should gain an understanding of the issues involved and of why specifying software is a difficult, but important process.

Requirements specification is a difficult, but important process...

Many practitioners in the field of software development would agree that software projects stand or fall on the requirements specification phase. Here are just two quotes:

[We] have grown to care about requirements because we have seen more projects stumble or fail as a result of poor requirements than for any other reason. (Kulak and Guiney, 2003)

It is estimated that getting the requirements right in the first place costs 50 to 200 times less than correcting code. (Cusumano and Yoffie, 1999)

1.1 The aims of this unit

In this unit we aim to:

- ▶ introduce you to the two activities involved in the process of requirements specification – elicitation and analysis (Section 2);
- ▶ describe the requirements document, which is the end product of this process (Section 3);
- ▶ allow you to practice the requirements specification process with two extended examples involving a hospital system and an online shopping system (Sections 4 and 5).

The two examples developed in Sections 4 and 5, referred to as the Hospital System and the Shopping System, will be used in the remainder of the course to illustrate other phases of software development.

1.2 Studying this unit

This unit contains a number of SAQs and paper-based exercises, which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

In Sections 2 and 3 of this unit most of your time will be spent reading, with the occasional paper-based exercise. We suggest that about half of your study time for this unit will be needed for Sections 2 and 3, with perhaps slightly more time being needed for Section 3.

Sections 4 and 5 have much less text to read, but the exercises are quite long and open-ended. We suggest that about half your study time will be needed for these two sections.

2

Eliciting and analysing requirements

In this section you will study:

- ▶ the role of analysts in the process of requirements specification;
- ▶ some of the techniques of eliciting requirements;
- ▶ how 'use cases' help to specify requirements.

2.1 The role of analysts

A large organisation may be made up from many different departments and each department, and person within it, has a part to play in the overall process of running the business. A **business system** is a combination of people and software working together to meet a particular set of business objectives. An expert in the business systems of the organisation called a **business analyst** (who may be a member of the organisation, or a consultant) is charged with understanding each part of each business systems and how the parts work together.

An effective organisation regularly reviews its goals as part of its planning cycles, and the business analyst will constantly analyse the business systems to ensure that they continue to meet the goals of the organisation. If the requirements of an organisation change, the new requirements might be met by changing the existing processes, but the business analyst might also identify a need for a new (or enhanced) software system in order to meet the new requirements.

Before a software development company (or the organisation's IT department) can start to create a software system it is the joint responsibility of the business analyst, the systems analyst (who is a technical expert in software systems) and the **requirements analyst** (who is able to elicit and specify requirements precisely) to define the exact requirements of the new software in a way that both the software developers and the client can understand. This process is called requirements specification, and it generally results in a document called the **requirements document** that defines a common understanding of what the new system will do.

You learnt about the systems analyst in *Unit 1*.

The main challenge for the systems analyst and the requirements analyst is to get to know and understand the **system domain**: the specific real-world context within which the new software system is required to operate. As the following two exercises illustrate, this can involve much explanation of the **business area** within which the system domain is located, and many questions and answers. This is typical of requirements specification.

Exercise 1

Consider the following situation. A consultancy company has approached an Open University (OU) tutor and offered to develop a spreadsheet to enable her to record the assignment marks for each of her students in her A227 tutor group. The tutor agrees. She says that she has a normal-sized tutor group this year, and there is the usual number of TMAs for a 30-point course.

For this exercise you will take on the role of a business analyst, who is familiar with the way OU courses are organised. What might you have to explain to the consultancy company's systems analyst, who is not at all familiar with OU courses?

Discussion.....

Here are some facts that the company might need to know. You may have thought of others.

- ▶ A tutor group is a group allocated to a single tutor. A group normally has around 20 students – sometimes a few more, sometimes a few less.
- ▶ Students are allocated to a group at the beginning of a year, but additional students can join, or withdraw from, the group at any time.
- ▶ TMA means tutor-marked assignment.
- ▶ 30 points is an indication of what fraction of a degree program A227 counts towards. A BA or a BSc honours degree usually requires 360 points of study.
- ▶ The usual number of TMAs for a 30-point course is four.
- ▶ Assignment marks are given as whole number percentages (integers between 0 and 100).
- ▶ A tutor might agree that an assignment can be handed in late.
- ▶ All students have unique personal identification numbers (PI numbers).

Exercise 2

Now take on the role of the systems analyst. All you know is that the tutor currently records TMA results on an OU-supplied paper form. What questions would you want to ask the business analyst to better understand the existing way in which tutors record TMA information?

Discussion.....

Here are some possibilities.

- ▶ Are the students listed on the form when the tutor gets it, or does the tutor have to enter them herself?
 - ▶ Does the completed form have to be submitted to the Open University, or is it just for the tutor's personal use?
 - ▶ What are the column headings on the form, for example is there a column for displaying the student's average TMA score?
 - ▶ Is the average mark for the group calculated and displayed for each TMA?
 - ▶ Does the form allow a tutor to identify a student by name, PI number (or both)?
 - ▶ Does the form allow the tutor to record any special circumstances for a particular TMA for a particular student, such as recording it was submitted late?
-

Although we have distinguished between a business analyst, system analyst and requirements analyst, in effect we have been talking about *roles* within the requirements specification process, and not necessarily job descriptions. In practice, particularly in smaller software companies or IT departments, the jobs may not be so clearly differentiated; an analyst (regardless of their job title) might undertake a combination of the roles described. Therefore, in the remainder of this unit, we will simply use the generic term **analyst** to refer to the person (or people) who undertakes the process of requirements specification.

2.2 What makes a good analyst?

Requirements elicitation, the process of determining the requirements of a new system, is a task that requires excellent communication skills because it involves extensive consultation with all the **stakeholders** – that is, anyone who has an interest in the system. The stakeholders include:

- ▶ the client – that is, the person or organisation commissioning the new system;
- ▶ the people involved with the client organisation, such as the eventual users of the system and those who will maintain it;
- ▶ those who will develop the new system, such as the various analysts, designers and programmers.

Requirements elicitation is the least technical phase of producing a software system. In the subsequent stages of software development specialised technical knowledge is required to produce different models of the new system (such as the sequence diagrams you saw in *Unit 1*), and code has to be written and tested. In contrast, requirements elicitation depends on social, communication and managerial skills. In particular, analysts need the following.

- ▶ Interviewing skills and listening skills, including the ability to ask appropriate questions and interpret what is said in response.
- ▶ Good interpersonal skills, to negotiate priorities and resolve conflicts amongst the stakeholders.
- ▶ The ability to observe, so that data gathered can be checked and validated, and new areas for questioning can be identified.
- ▶ Strong organisational skills, to collect and analyse a large amount of information.
- ▶ The ability to critically evaluate information gathered from multiple sources.
- ▶ The ability to *analyse* (sometimes high-level information has to be decomposed into detail) and *abstract* (sometimes low-level details need to be brought together to develop a more general understanding).
- ▶ The ability to communicate their understanding of the requirements to the stakeholders, which may require that they can present complex information not only in written form, but in ways that the stakeholders understand, such as graphs or existing organisational formats.

As you can see, the job of the analyst is not an easy one, however it is essential that requirements elicitation is a thorough process, because misrepresenting the requirements may lead to the development of software that is unsuitable for the needs of the client organisation.

Despite the many skills required of an analyst, because it is less technical it can be tempting to leave requirements specification to a non-specialist in the client organisation. This is dangerous because non-specialists may impose their own bias on the requirements, as the following cartoon shows.

Requirements elicitation may be referred to simply as elicitation.

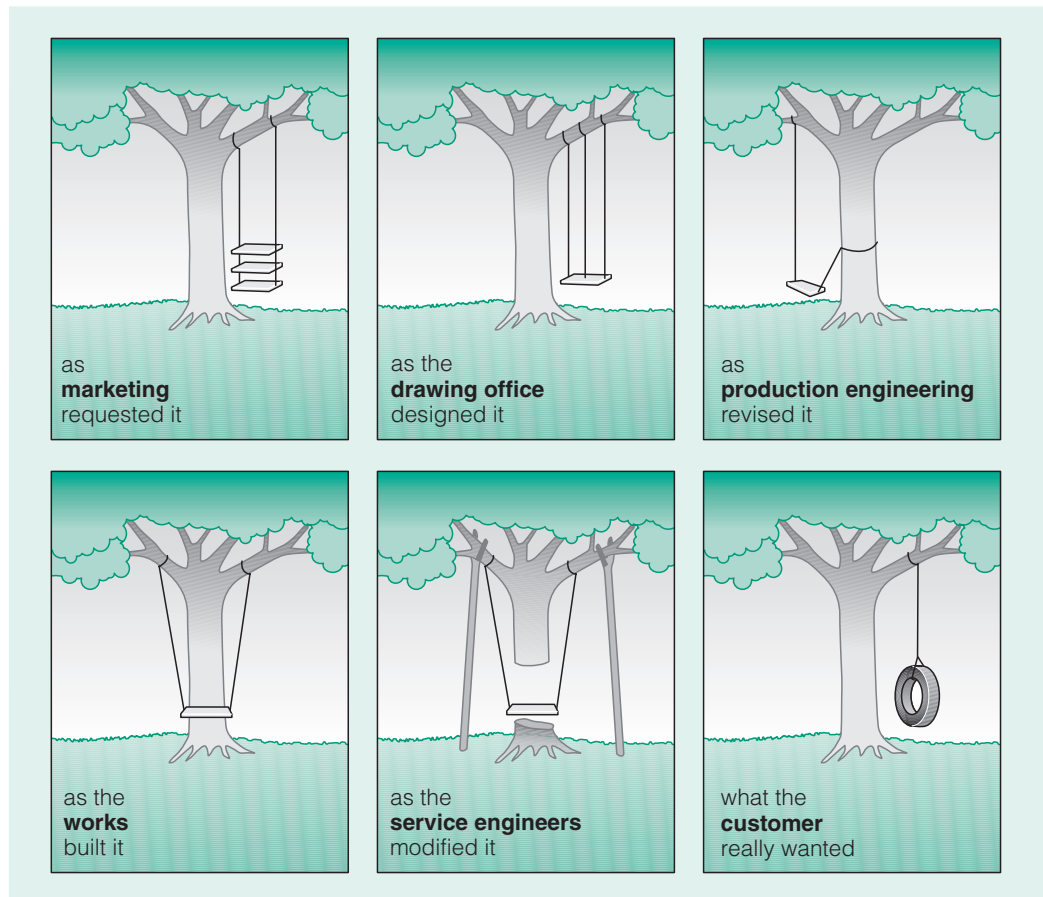


Figure 1 The challenge of specifying requirements

On a more serious note...

Unfortunately, there are many well-catalogued examples of real-world failures in requirements specification.

In 1993, after spending over half a billion dollars on it, the London Stock Exchange scrapped its "Taurus" project (intended to control the "Transfer and Automated Registration of Uncertified Stock"). It had been Europe's biggest new software undertaking. What went wrong?

The problem was failure to do an adequate analysis of requirements. Requirements for the project were not clearly defined, and they were changed constantly by the myriad of players in the project. "Share registrars, anxious to protect their jobs, demanded a redesign of the system to mimic today's paper-shuffling. Scrapping share certificates called for 200 pages of new regulations. Computer security, with all messages encrypted, went over the top. Companies' insistence on the 'name on register' principle, which allows them to identify their shareholders instantly, made design harder. And so on." [*Economist*, "When the bull turned", 1993, p. 81]

Hay (2002, p. xxvii)

It is also important to realise that organisations are rarely static and requirements may change during a software development project. A good analyst will try to anticipate this and manage change by deciding distinct points at which changes will be considered. However, every change in requirements *after* the requirements specification phase costs time and money, and generally the later in the project a change is made, the more expensive it is, so, as far as is possible, the analyst has to try to minimise late changes.

2.3 Techniques used to elicit requirements

There are many techniques for determining the requirements of a new system but all of them involve a high degree of interaction between the analyst and the other stakeholders. Traditional methods of requirements elicitation involve asking questions (through interviews, questionnaires, surveys, group discussions, etc.), making observations, and studying documentation of the existing processes.

Sometimes a prototype system is used. This is a demonstration system constructed so that the stakeholders can visualise the system (or part of it), and the analyst can obtain their feedback. However, there are disadvantages to the use of prototypes. It can be difficult to convince the stakeholders that the prototype system is not a complete system – once the stakeholder has seen the prototype working they may expect to see the finished version appear very quickly afterwards.

Different methods for eliciting requirements

Many new approaches to software development, such as Rapid Application Development (RAD) and eXtreme Programming (XP), make use of prototypes as a means of requirements elicitation.

Another technique, Joint Application Development (JAD), has been around since the late 1970s and has recently enjoyed a revival. In JAD all the stakeholders meet in a series of workshops, which capitalise on group dynamics in order to develop requirements.

You will learn more about RAD and XP in *Unit 14*.

2.4 Asking questions

Asking questions is fundamental to understanding the business area and determining what the new system will do – in other words, what its behaviour must be.

In some cases the analyst works with a single named person within the organisation, most often the business analyst or an IT contact, and this person will coordinate with the other stakeholders to determine their requirements. This model has the advantage that there is a single person always available to answer questions. The downside is that critical information may be communicated via a number of people, and may get distorted in the process.

Another approach is for the analyst to elicit the information directly from individual stakeholders, or groups of stakeholders. In either case it is important, when questioning, that the analyst uses the stakeholders' business language (i.e. the 'jargon' common in their business area). This ensures that the stakeholders know what is being asked and thus reduces misunderstandings. One way of achieving this is for the analyst to build up a glossary of terms although, since terms can vary from department to department, they must still take care that the correct information is being discussed.

When collecting information an iterative process is often used. To begin with, broad questions are asked. The answers to these questions are studied and then other questions are formulated, many of which are more detailed. The answers to these questions are further studied and even more detailed questions formulated. The process continues until the analyst and the client are happy that they understand the behaviour required of the system.

Scope

A common approach to requirements elicitation is to start by asking questions about the **scope** of the system; that is, determining the breadth of services that the system is expected to provide. For example, if you are developing a system for a hotel, a question that you might ask the client is:

Should the system deal with billing guests, or is there a separate system for dealing with billing?

It is equally important to define what is outside the scope of the system. For example, it may be decided that a complex requirement, such as dealing with complaints about billing is better left to a solely human process.

Behaviour

Once the scope of the system has been determined, an analyst will try to determine (within that scope) what the behaviour of the system should be. One approach to doing this is to consider what tasks the users want the system to deal with. For example, in the hotel system described earlier the analyst might, through questioning, determine that the hotel receptionist will be the main user, and that the main tasks the receptionist might need to perform are checking in a guest, checking out a guest and listing all guests in the hotel.

Once the tasks have been identified, the analyst will then ask more detailed questions about each of them. For example, consider the task:

The hotel system should be able to check in a guest.

Such a statement gives rise to a number of questions:

- 1 What does 'check in' mean? Does it mean that when a guest arrives the system records their personal details?
- 2 What information does the receptionist have to provide when a guest checks in: would name, address, length of stay and credit card number be enough?
- 3 Does the system allocate a room to the guest checking in?

All these questions are about specific tasks that the system must be able to perform; that is, its **behavioural requirements**.

The first relates to what the system has to record, the second relates to what information has to be input to the system to achieve the task and the third relates to what the system might output.

In this simple example we have assumed that the data input and output is on a computer screen. However data input and output can come in a variety of formats. For example, files, other computers or from pieces of electronic equipment such as the bar-code readers found in a supermarket.

Exercise 3

What questions might the analyst need to ask about the following tasks?

- (a) The hotel system should be able to check out a guest.
- (b) The hotel system should be able to list guests in the hotel.

Discussion.....

Here are some examples

- (a) What information needs to be provided to the system by the receptionist when a guest checks out?

Do records have to be kept about the stay? If so, what should the system record about the guests after they have checked out?

Is generating a bill part of the check out process, or a separate process that can take place at another time?

(b) What information is required on the list of guests?

Is the list simply a snapshot of the situation that exists at the time the list is requested, or might the receptionist be required to provide a time and date to obtain a list from some previous day?

Exercise 4

Suppose a client has told you that a new library system should:

Allow a borrower to check out library books.

What questions might you, as an analyst, ask to clarify and refine this task?

Discussion.....

There are lots of appropriate questions. Here are just a few.

- ▶ Does a borrower have to be authenticated as a member of the library before the check out process? If so how is that done? What happens if the borrower is not a member of the library?
- ▶ Are there any circumstances under which an authenticated borrower is not allowed to borrow a book (e.g. a limit on the number of books that can be checked out)? If so, what happens in these cases?
- ▶ What is recorded when a book is loaned to a borrower?

2.5 Analysis and specification of use cases

Alongside the elicitation of requirements, the analyst has to analyse the information gathered about the requirements of the system, and extract and clarify the important parts to form a definitive set of behavioural requirements. The process of analysing and distilling the information from the requirements elicitation process into a clear and coherent description of the system and its behaviour is called **requirements analysis**. To some extent, the requirements elicitation and requirements analysis phases are intertwined, with each informing the other through a process of feedback and iteration between the analyst and the stakeholders. As the analyst elicits information about the system, and then analyses it to extract the tasks it must perform, it will lead them to ask more pertinent and detailed questions. Through this iterative process, the analyst removes ambiguities and ensures that their view of the behaviour of the system is the same as that of the client. The final goal is to describe each task the system must perform in concise English that is unambiguous and understandable to both the client and the software developers. Such a description is referred to as a **use case**.

A use case for the check in task for the hotel system might be as follows:

Check In Guest. The receptionist provides the system with the guest's name and address, and passport number (if the guest is not British), and identifies the arrival date and the departure date. The system allocates the guest to a room. The system records the guest's details, the arrival and departure dates, and that the guest has been assigned to the room.

To be useful, the behaviour of a system must be specified at the right level of detail. Aspects of the behaviour that are important to the client and users must be easily identifiable and should not be hidden by too much detail. On the other hand, if there is not enough detail, there is a risk that the real requirement will be misunderstood by the software designers. It is part of the professional skill of the analyst to be able to write requirements to an appropriate level of detail.

As an example, consider this statement, which describes one aspect of the behaviour required of a new system.

The system should provide financial information for the company accountant.

Such a statement would seem to hide a number of more clearly defined tasks – further requirements elicitation would be required to identify more specific behavioural requirements, with individual tasks clearly identified. As a result, the following tasks might be identified.

- ▶ The system should display the total receipts for a given day.
- ▶ The system should display the total profits, broken down by department, for a given year.

These can then be written as two separate use cases.

- 1 **List Receipts.** The administrator identifies a day. The system displays the total receipts for that day.
- 2 **List Profits.** The administrator identifies a year. For each department the system displays the name of the department and the profit made by that department in that given year.

Exercise 5

Consider the task:

The hotel system should be able to list guests in the hotel.

Here is a snippet from an interview with a hotel manager. Read it and then write the *List Guests* use case.

Analyst: So at any moment in time you need a list of current guests checked into the hotel?

Hotel Manager: Yes

A: Will you ever need a list for any other particular day, or is it really just a snapshot of whenever the list is generated.

HM: It's just a snapshot of a given moment in time so all guests can be accounted for if there is an emergency of some sort.

A: So if a guest has already checked out even a moment before the list is generated, they don't appear on the list?

HM: That's right

A: And what information is required on the list?

HM: Just the name of each guest and their room number.

Discussion.....

The *List Guests* use case is as follows.

List Guests. The system displays the name and room number of each guest currently checked into the hotel.

Exercise 6

Consider the following task.

The hotel system should be able to check out a guest.

Here is a snippet from an interview with a hotel manager. Read it and then write the *Check Out Guest* use case.

Analyst: What information should be provided to the system by the receptionist?

Hotel Manager: The room where the guest was staying.

A: Not the name of the guest?

HM: No, just the room.

A: Is generating a bill part of the check out process or a separate process that can happen at other times?

HM: It is a separate process dealt with by a separate billing system

A: And what should the system record about the guest who has just checked out?

HM: Nothing

A: You mean you don't keep any record once a guest has left the hotel?

HM: Not on this system. The billing system keeps records, of course.

Discussion.....

The *Check Out Guest* use case is as follows.

Check Out Guest. The receptionist identifies the room of the guest checking out. All information relating to that guest is removed from the system.

Exercise 7

Consider the following task.

The library system should allow a borrower to check out a library book.

Here is a snippet from an interview with a librarian. Read it and then write the *Borrow Book* use case.

Analyst: Does a borrower have to be authenticated as a member of the library before a book can be checked out? If so how is that done?

Librarian: Yes they have to be authenticated. The librarian must first check that the borrower's library number is valid; if not, they are given a message.

A: So each borrower has a unique number? Do they have a card with it on?

L: Yes each number is unique, and the borrower's card has a barcode that stores their number.

A: And what happens if they are not authenticated?

L: The librarian is given the message that they can't check out books.

A: Is there a limit to how many books a borrower can check out? If so what should happen if they try to check out too many books?

L: Yes, each borrower has a limit on the number of books that they can check out. If they try to check out more books than their limit we should be given a message to say that the limit has been reached.

A: Are there any other circumstances under which an authenticated borrower is not allowed to borrow a book? If so what are they, and what happens in these cases?

L: Yes, if the borrower is in debt to the library (they have an unpaid fine). If this is the case the librarian should be given a message to say that they can't check out more books unless they pay the fine.

A: And what is recorded by the system when a book is loaned to a borrower?

L: Just that the borrower has borrowed the book and the date of return.

Discussion.....

The use case for *Borrow Book* is.

Borrow Book. The librarian identifies the borrower. If the borrower is not authenticated, or the borrower has reached their limit on loaned books, or the borrower is in debt to the library then the librarian is given an appropriate message.

If the borrower is authenticated, has not reached their limit, and is not in debt, the librarian identifies the book, and the system records that the borrower has borrowed the book and the date of return.

2.6 Requirements for user interfaces

The discussion in this section has concentrated on how the analyst, using a knowledge of the system domain coupled with the techniques of requirements elicitation and requirements analysis, arrives at an understanding of the system behaviour which is specified in terms of use cases.

Although the information input to and output from the system has been specified, nothing yet has been mentioned about the details of *how* the users will actually interact with the system, for example exactly how the user identifies a borrower. User interface design is a specialised area, and will be considered later in *Units 11, 12 and 13*.

3 The requirements document

The process of requirements specification culminates in a requirements document that should provide both the client and the software developers with a clear and unambiguous understanding of what tasks the new software system is expected to perform, and what the limitations on the software developer are.

The formation of the requirements document is a pivotal stage in the development of a new software system as the document provides the base on which the rest of the project will proceed – a kind of handbook that defines a firm foundation for all aspects of the development of the new system.

Once the requirements document has been approved by both the client and the software company, it often also constitutes a legal contract, defining what the client has agreed that the software developers will produce.

In this section you will study in more detail what is contained in a requirements document.

3.1 A template for a requirements document

There are many commercial templates available for requirements documents. Here is one example, which assumes an object-oriented development method.

1 Project outline

- 1.1 The business context
- 1.2 The system domain
- 1.3 The stakeholders

2 The behavioural requirements

- 2.1 The scope of the problem
- 2.2 The use cases
- 2.3 Data requirements

3 The constraints

- 3.1 Interface requirements
- 3.2 Performance requirements
- 3.3 Security requirements
- 3.4 Implementation requirements
- 3.5 Political and legal requirements

4 Acceptance tests

5 Project management

- 5.1 Schedule
- 5.2 Budget

There is no one format all requirements documents have to conform to but one should always contain:

- 1 a brief description of the part of the business within which the new software system will operate, that is the system domain;
- 2 a description of the behavioural requirements that the system must meet (in the form of a series of use cases);
- 3 a description of the **acceptance tests** that will be performed to assure the client that all the agreed behaviour is provided by the completed system.

As regards the requirements document itself, this course will focus on only these three aspects; it will not consider the wider business context of a system, or project management (these topics can form whole courses by themselves). Subsection 3.3 of this unit will, however, briefly consider what is meant by a constraint on a software system because it can sometimes be a challenge to distinguish between constraints and requirements. It is important to understand that constraints can never be ignored in real-world projects, but the simplified examples of software development in the remainder of this course will generally exclude them.

SAQ 1

Which sections of the example requirements document covers the three aspects, system domain, use cases and acceptance tests, described above.

ANSWER.....

Part 1.2 contains the information about the system domain.

Part 2.2 covers use cases.

Part 4 deals with acceptance tests.

3.2 The main sections of a requirements document

Here we will briefly consider the three sections of a requirements document, identified above, that will be important in the remainder of this course.

System domain

The system domain section of the requirements document contains a brief description of the part of the real world within which the proposed system will operate. It should contain only information that is within the scope of the new system, and it generally does not contain a lot of detail about the behaviour of the system. For example, if a hotel chain was interested in a system to check in small parties of guests, the system domain section might read as follows.

Each hotel has a number of rooms, each of which may be empty or have up to three guests staying in it. Each room has a unique number. A hotel has a reception department which is responsible for recording information about parties of guests and the rooms that they are in. A party may consist of up to ten guests. Only guests in the same party can share a room.

Use cases

The behavioural requirements of the system define *what* the completed system will do. These are often couched in terms of a set of use cases that have emerged from the requirements analysis process as described in Subsection 2.5.

Acceptance tests

Testing is a very important aspect of software development, and typically takes place iteratively and at several stages throughout the software development process.

Acceptance testing, which aims to convince the client that the system works as specified, is a vital component of testing. As each use case is drawn up and refined for inclusion in the requirements document, the client and the analyst devise acceptance tests that will be applied to the complete system to demonstrate to the client that the use case has been satisfactorily met; that is, that the system exhibits the required behaviour. If the completed system fails any acceptance tests both parties will know that it is not implementing all the agreed behaviour as outlined in the requirements document. This may mean that the client will not accept the final product, as it does not meet the terms of the contract as represented by the requirements document.

Part of the task of drawing up acceptance tests is to identify what should occur as the system is assessed with respect to its requirements. We will take a very simple approach to drawing up acceptance tests, simply giving a statement of the main aspects that should be checked for each use case.

For example, suppose the following is a use case for a library system.

List Loans. The librarian identifies the borrower. The system displays a list of the books that the borrower currently has on loan.

What acceptance tests might be needed in order to check that this requirement is met? Situations need to be identified in which the behaviour of the system might vary. Well, either the borrower (here we assume the borrower has been authenticated) has books on loan or not, and so two distinct possibilities need to be tested.

- 1 Test that the list is produced correctly if the borrower has no books on loan.
- 2 Test that the list is produced correctly if the borrower has books on loan.

You have seen that use cases are often refined in an iterative way whilst they are being developed. The acceptance tests would be refined at the same time. For example, consider this refined use case.

List Loans. The librarian identifies the borrower. The system displays a list of the books which that borrower currently has on loan, and indicates any that are overdue.

The acceptance tests might now be refined as follows.

- 1 Test that the list is produced correctly if the borrower has no books on loan.
- 2 Test that the list is produced correctly if the borrower has books on loan and there are no overdue books.
- 3 Test that the list is produced correctly if the borrower has books on loan and there are some overdue books.

There are generally many different rounds of testing of parts of the system, and of the system as a whole, before the acceptance tests are finally carried out. You will learn more about testing in *Units 10* and *13*.

Exercise 8

Here is another use case from the library system.

Borrow Book. The librarian identifies the borrower. If the borrower is not authenticated, or the borrower has reached their limit on loaned books, or the borrower is in debt to the library then the librarian is given an appropriate message.

If the borrower is authenticated, has not reached their limit and is not in debt, the librarian identifies the book, and the system records that the borrower has borrowed the book and the date of return.

If you were the client, what acceptance test(s) would you require to make sure that the system fulfilled this requirement under normal circumstances?

Discussion.....

The following four acceptance tests would be appropriate:

- 1 Test that an attempt by an unauthenticated borrower to borrow a book results in an appropriate message.
- 2 Test that an attempt by an authenticated borrower, who has reached their loan limit, to borrow a book results in an appropriate message.
- 3 Test that an attempt by an authenticated borrower, who is in debt to the library, to borrow a book results in an appropriate message.
- 4 Test that a new loan is recorded correctly when the borrower is authenticated, not over their loan limit and not in debt to the library.

3.3 Constraints

Many books refer to constraints as 'non-functional requirements' and behavioural requirements as 'functional requirements'.

An important section in the requirements document is that devoted to listing constraints on the new system. A **constraint** is a limitation on the system or its development. For example, there may be a need to maintain the style of an existing graphical user interface, or the development company may constrain its developers to use existing software development tools. There could be constraints on performance (the system's reliability, throughput, etc.) or security (restricted access to information). Often an organisation will have company standards covering the style in which code is written or documented (to facilitate maintenance) and so any new software must comply with these standards.

Constraints may also occur as a result of external circumstances. For example there may be legal and security constraints imposed by the government, or other external agencies. A client may have to place restrictions on what data can be made available and to whom (in order to comply with the UK Data Protection Act), or to ensure that the system is accessible by disabled people (the UK Disability Discrimination Act). An increasingly important consideration, and another source of constraints, is maintaining the security of data as it is transmitted across networks.

Consider the following initial description of a proposed system for a bus company, which contains some constraints.

The company requires a system that can run on a PC with Windows XP installed. The software should be written to the company's standards. In response to customer queries the system must be able to perform a number of tasks. These include being able to provide the route number and final destination of all routes passing through a given stage, and determining the fare between any two stages. The system must also be able to provide a timetable when given a

starting stage and a stopping stage, and a day of the week. There is a limit of 512 megabytes of memory and ten gigabytes of hard disk space available for this system, and the response time for any query should be less than five seconds.

This description indicates various ways in which the developers will be limited or constrained in their choices as they develop the system, that is:

- ▶ the software must run on a PC operating Windows XP;
- ▶ the software must be written to the company's standards;
- ▶ there is a limit of 512 megabytes of memory and ten gigabytes of hard disk space;
- ▶ a response time of less than five seconds must be achieved.

Distinguishing between behavioural requirements and constraints

An analyst must be able to distinguish between behavioural requirements and constraints; that is, between what a system should be able to do, and limitations on how it should do it.

Exercise 9

Consider the following requirements of a system for storing the route (flight plan) of a flight between two airports. Classify each as a behavioural requirement or a constraint.

- (a) The system should periodically update the aircraft position file.
- (b) The response time of the SCAN command should be no more than 0.2 milliseconds.
- (c) The flight coordinates should be held in a sorted collection.
- (d) An object-oriented language should be used.
- (e) When given an aircraft identification number, the system must display the aircraft's position.

Discussion.....

- (a) This is a behavioural requirement; it describes a task the system must perform.
 - (b) This is a constraint: it imposes a limitation on the system.
 - (c) This is a constraint: it constrains the developer to use a particular type of data structure.
 - (d) This is a constraint: it constrains the developer to a particular category of programming languages.
 - (e) This is a behavioural requirement: it describes a task the system must perform.
-

Distinguishing between behavioural requirements and constraints is not always straightforward. A client, not knowing or even caring about the distinction between them, may mix information about what a system should do (its behaviour) and how it should do it (the constraints). For example, here is a report from a person in the client organisation to an analyst:

When a guest checks in at the hotel the receptionist should enter the name and address of the guest, along with their arrival date and how long they are staying, and their allocated room number. This information needs to be recorded by the system, but it should be deleted when they check out. Our hotels all have PCs that are networked via a Linux server, and so the system needs to be compatible with this. The graphical user interface for the new system must have the same look and feel as the existing graphical user interface because our receptionists

find this easy to use. The receptionist must, at any time, be able to produce a summary report that contains the names of all the guests staying at the hotel, along with their room numbers. This report should take no more than five seconds to generate as it may be needed in case of a fire.

The behavioural requirements of the system can be summarised as follows.

- ▶ Record a guest who is checking in.
- ▶ Delete all information about a guest who is checking out.
- ▶ List all guests in the hotel.

The following are constraints on the system.

- ▶ The system must be compatible with PCs networked via a Linux server.
- ▶ The graphical user interface must have the same look and feel as the existing graphical user interface.
- ▶ The summary report must take no longer than five seconds to generate.

Exercise 10

The following is a short statement that has been provided by a client as the starting point for the requirements elicitation process. Summarise the behavioural requirements and the constraints.

The monitoring system should monitor and display the temperatures in a series of reactors, detect any hazard conditions from these temperatures that may indicate a reactor malfunction, and periodically write temperature data to a database that is used to provide information about the day-to-day running of the reactors. Ten megabytes of memory is available for this system. All the file handling that the system performs should be implemented by means of calls to the host machine's operating system. The system should also periodically archive the temperature database to some back-up storage device.

Discussion.....

The behavioural requirements can be summarised as follows.

- ▶ Monitor the temperatures in a series of reactors.
- ▶ Display the temperatures in a series of reactors.
- ▶ Detect any hazard conditions from the temperatures that may indicate a reactor malfunction.
- ▶ Periodically write temperature data to a database.
- ▶ Periodically archive the temperature database to a back-up storage device.

The constraints can be summarised as follows.

- ▶ The system must fit into ten megabytes of memory.
 - ▶ All the file handling that the system performs should be implemented by means of calls on the operating system.
-

The analyst should aim for no more constraints than are needed for the business purpose of the system, negotiating with the client to determine strictly necessary constraints. Unnecessary constraints may lead to limitations on the choices at later stages in the software development process. For example, it is not always clear at the beginning of a project in which language it would be best to implement the system. If possible, it is best for the client and the analyst *not* to make assumptions that a particular language will be used; the software developer will have a better idea of the software

structures needed for the system, and consequently will be able to choose the most appropriate language.

3.4 The requirements document as a contract

In the contract between a client and a software developer there is normally a clause stating that the requirements document must be agreed by both the client and the developer and that, once this agreement is reached, no more changes may be made to it without the permission of both parties. This means that the client must thoroughly check the requirements document and ensure that it represents the system they require. Once agreement is reached, the document is formally 'signed off' by the client, indicating that it represents the agreed system. It is common for a percentage of the developer's fees to be paid at this point.

It is very important to have a point at which the requirements are agreed by both parties because of the danger of **requirements creep**: a phrase that describes the, all too common, situation where the software developers come under pressure to deliver more than was originally agreed. However, it must also be acknowledged that however carefully it has been prepared, it may not be possible to guarantee that the requirements document contains a complete and fixed set of requirements.

It is possible there may be external factors (new government legislation, for example) that mean that, through no fault of the organisation or analyst, the requirements have to be modified after the requirements document has been agreed. It is important that both the client and the analyst acknowledge the possibility that the requirements might change, and have some contingency plans to manage any changes in requirements.

In practice this can be done in a number of ways. For example, there might be an agreement that, once the requirements document has been signed off, any changes must be dealt with via an additional work request, which has to be paid for by the client. This ensures that the developers get paid for any additional work to that specified in the requirements document, and it focuses the mind of the client to avoid incurring any extra financial penalties.

Alternatively, it might be agreed that any changes to requirements must be jointly managed by the project manager and the analyst. Managing change at a high level prevents other stakeholders from being able to change their minds easily about requirements, although this approach requires strong project management. Under these circumstances only the most essential changes will be implemented, with others being either shelved or noted for any future development work.

In this section, you have learnt about the requirements document. You also learnt to distinguish between behavioural requirements and constraints.

4

The Hospital System

In this section you will be given a brief description of part of a proposed software system to deal with the admission and treatment of patients in a hospital (the Hospital System). You will then go through requirements elicitation and analysis exercises in order to produce a requirements document detailing:

- ▶ the system domain;
- ▶ the use cases;
- ▶ the acceptance tests.

4.1 The starting point

As a starting point an administrator has produced the following brief initial description of how the admission and treatment of patients is dealt with in the hospital. As an analyst you will use this statement as the starting point for your requirements elicitation and analysis.

The hospital has several wards divided into male wards and female wards. Each ward has a fixed number of beds. When a patient is admitted they are placed onto an appropriate ward.

The doctors in the hospital are organised into teams such as Orthopaedics A, or Paediatrics, and each team is headed by a consultant doctor. There must be at least one grade 1 junior doctor in each team.

The Administration department keeps a record of these teams and the doctors allocated to each team. A patient on a ward is under the care of a single team of doctors, with the consultant being the person who is responsible for the patient. A record is maintained of who has treated the patient, and it must be possible to list the patients on a ward and the patients treated by a particular team.

4.2 Asking questions

In this section you are asked to bear in mind that the system being considered is not intended to be anything like as complex as it would be in real life. It is assumed that the remit is to construct a prototype that may subsequently be used in developing a far more sophisticated system. So, although the questions are similar to those that would be asked during requirements elicitation and analysis, the answers will be comparatively simple.

One of the activities mentioned is recording the admission of a patient. To gain more detail about this task a hospital administrator could be questioned – consider the following example dialogue.

Analyst: What information do you need to record about patients when they are admitted to the hospital?

Hospital Administrator: Well, their name and sex. Oh, and their age. No, not their age, their date of birth.

A: Don't you record what's supposed to be wrong with them?

HA: No, all we need to do is to note which team will take care of them.

A: What team is this?

HA: Each patient is under the care of a particular team of doctors. We are told which one when the patient is admitted.

A: And how do you identify a team?

HA: They all have a team code, for example A&E or Paediatrics.

A: OK, and do you also decide which ward to admit the patient to?

HA: We'd like the system to tell us that, based on which wards have free beds and whether the patient is male or female.

A: Do all wards have the same fixed number of beds?

HA: No, some have a greater capacity than others.

A: And so what should happen if there is no bed available on a ward of the right sex?

HA: Well the administrator should get a message that the patient cannot be admitted, but generally there are several appropriate wards with free beds.

A: How should the ward be chosen in that case?

HA: The patient should be admitted to the ward that has the largest number of vacant beds.

A: OK, so suppose that a bed is found on an appropriate ward for a patient, what then needs to be recorded?

HA: Well the patient's personal details, like I said earlier, and the team that will look after them.

A: Anything else?

HA: Oh, yes, the consultant that is responsible for the patient.

A: And which consultant is this?

HA: They head the team that cares for the patient.

A: Does this mean that all the consultants on a team are responsible for the patient?

HA: No, you don't understand. There is only one consultant on each team. The consultant is the leader of the team. I thought I'd made that clear in my description.

A: Well, yes, it did say that each team was headed by a consultant. I just hadn't realised that there weren't any other consultants on the team.

HA: Yes, the rest of the team is made up from junior doctors.

A: Right, and how many junior doctors might there be on a team?

HA: There isn't any upper limit, but there has to be at least one grade 1 junior doctor.

A: And how many grades are there?

HA: Just three, grades 1, 2 and 3.

A: OK, I think I've got that.... So let's just check. When a patient is admitted you need to note their name, sex and date of birth, as well the team that cares for the patient and the consultant that is responsible for the patient. Is that all?

HA: Yes, and then the system should inform the administrator of the ward that the patient has been admitted onto.

These initial questions give a lot more detail about what the system has to do when a patient is admitted; you should now have a clearer idea of what is involved and how the information involved is structured. As an analyst you would now proceed to ask more questions to identify other tasks to be performed by the system.

Exercise 11

Read the initial description of the Hospital System above and come up with some questions that you might ask in order to determine the scope of the system, and the behavioural requirements. You should not spend more than fifteen minutes on this very open-ended exercise so, in the first instance, try to think of questions that might allow you to identify three or four specific tasks, and then to clarify the requirements for these tasks.

Discussion.....

There are many possible answers to this exercise. After a few questions about scope we have started to identify different tasks and to ask questions to clarify those tasks. However, this is not a definitive list, and the answers are just plausible ones that we have made up.

The answers given will help draw up the system domain description and the use cases for the requirements document.

Analyst: So let me just check what this system must do. You need to admit patients and discharge them, and you also need the system to be able to output various lists?

Hospital Administrator: Yes, oh and also to record who has treated a patient.

A: So the system needs to know about doctors and consultants too?

HA: Yes that is right.

A: Let me just get this clear, does this system also need to be able to record what treatment a patient has received?

HA: No, that is done by a separate medical records system, not by this system.

A: OK, so let's look at something specific to this system. You initially need to admit a patient to a ward, which we talked about earlier, but does a patient always stay on the same ward?

HA: No, not necessarily, sometimes we transfer patients between wards.

A: What information needs to be provided when this happens?

HA: The administrator has to identify the patient, and then the ward they are being transferred to.

A: Presumably the new ward has to be of the right type?

HA: Of course, and there must be room on it too.

A: And what is recorded when a patient is transferred?

HA: That the patient is on the new ward.

A: Is there any need to record that they used to be on a different ward?

HA: No, not by this system.

A: You mentioned some lists; tell me about what kind of information the system needs to output?

HA: We need a list of all the patients on a particular ward, and a list of all the patients cared for by a particular team.

A: Let's take the first one first. What does that involve?

HA: The administrator identifies a particular ward and the patients' names and ages should be displayed. Perhaps the ward could be identified by selecting from a list of ward names?

A: We'll bear that in mind, but we'll leave the details of the user interface until later. How about the second list?

HA: Well the administrator identifies a team and all the patients cared for by that team should be listed.

A: And what do you need on this list?

HA: The patient's name and the name of the ward that the patient is on.

A: So let's move on to what you need to do to record treatment. Do you need to know every doctor in a team who has actually treated the patient?

HA: Yes, we need the system to record this.

A: What exactly does this entail?

HA: Well the administrator has to identify the patient, and the doctor (consultant or junior doctor) who has treated the patient, and the system should record that the doctor has treated the patient.

A: But only doctors in the team that the patient is allocated to can treat the patient.

HA: Yes, that is right.

A: OK, do you ever need to list exactly who has treated the patient?

HA: Yes, I forgot about that. For any particular patient we need to know the name of the consultant responsible for the patient, and the code of the team that is caring for the patient, as well as the name of each doctor who has treated the patient, and their grade if they are a junior doctor.

A: Do you really need all of this information on this list? Surely if you know the code of the team you could have the system look up the name of the consultant doctor?

HA: Yes that is true, but it would be convenient to have it on the list.

A: So tell me what happens when a patient leaves the hospital?

HA: Well we have a discharge process, and the discharged patient should be removed from this system.

A: So what does the administrator do?

HA: Oh she should just tell the system which patient is to be discharged, and all information relating to that patient should be removed from the system.

A: So you don't need any record of a patient once they are discharged?

HA: Well of course we keep their medical records, but not on this system, that is a totally different system.

Exercise 12

Write down the tasks that can be identified from the dialogue contained in the discussion to Exercise 11.

Discussion.....

In the order they emerge from the dialogue:

- 1 Record the treatment of a patient by a doctor;
 - 2 Transfer a patient from one ward to another;
 - 3 List the patients on a ward;
 - 4 List the patients under the care of a team, with their wards;
 - 5 List the doctors who have treated a patient, and the patient's team and consultant;
 - 6 Record the discharge of a patient.
-

Exercise 13

From the dialogue in the discussion to Exercise 11, write the *Transfer Patient* use case that describes what the system must do when a patient is transferred between wards.

Discussion.....

Transfer Patient. The administrator identifies the patient and the new ward. The new ward should be of the appropriate type and have at least one free bed. The system records:

- ▶ the removal of the patient from their current ward;
 - ▶ the allocation of the patient to the new ward.
-

Exercise 14

The following use case has been identified from the analysis. What acceptance tests would you require in order to ensure that this requirement was met?

Admit Patient. The administrator provides the system with the patient's name, sex and date of birth and identifies the team that cares for the patient.

The system identifies a ward of the appropriate type with empty beds; if there is a choice of such wards one of those with the largest number of empty beds is chosen.

If there is no appropriate ward with empty beds the administrator is informed of this fact.

If there is an appropriate ward with empty beds the system does the following two things.

- 1 It records:
 - ▶ the patient's name, sex and date of birth;
 - ▶ that the patient is under the care of the given team;
 - ▶ that the consultant that heads the team is responsible for the patient;
 - ▶ that the patient is on the ward.
- 2 It informs the administrator of the ward to which the patient has been admitted.

Discussion.....

Either there is a suitable ward available or not. Therefore there are two possibilities to test for, as follows.

- 1 Test that, when there is an appropriate ward available:
 - ▶ the admission of a patient is recorded correctly;
 - ▶ the administrator is informed of the ward to which the patient has been allocated.
- 2 Test that, when no appropriate ward is available, an attempt to record the admission of a patient results in the administrator being informed that there is no appropriate ward available.

4.3 The requirements document

After completing the requirements elicitation process and the analysis of the responses, the system domain, use cases and acceptance tests sections of the requirements document may look like the following.

The requirements document for the Hospital System

This requirements document is reproduced in Section 3 of the *Handbook*.

System domain

In the hospital, there are a number of wards, each of which may be empty or have on it one or more patients. There are two types of ward, male wards and female wards. A ward can only have patients of the specified sex on it. Every ward has a fixed capacity, which is the maximum number of patients that can be on it at one time (i.e. the capacity is the number of beds in the ward). Different wards may have different capacities. Each ward has a unique name. The hospital has an Administration department that is responsible for recording information about the hospital's wards and the patients that are on each ward.

The doctors in the hospital are organised into teams, each of which has a unique team code (such as Orthopaedics A, or Paediatrics). Each team is headed by a consultant doctor who is the only consultant doctor in the team; the rest of the team are all junior doctors, at least one of whom must be at grade 1. Each doctor is in exactly one team. The Administration department keeps a record of these teams and the doctors allocated to each team.

Each patient is on a single ward and is under the care of a single team of doctors; the consultant who heads that team is responsible for the patient. A patient may be treated by any number of doctors but they must all be in the team that cares for the patient. A doctor can treat any number of patients.

Use cases

The Hospital System will provide support for the administration of the hospital by allowing hospital administrators to do the following.

- A Admit Patient.** The administrator provides the system with the patient's name, sex and date of birth and identifies the team that cares for the patient.

The system identifies a ward of the appropriate type with empty beds; if there is a choice of such wards one of those with the largest number of empty beds is chosen.

If there is no appropriate ward with empty beds the administrator is informed of this fact.

If there is an appropriate ward with empty beds the system does the following two things.

- 1 It records:
 - ▶ the patient's name, sex and date of birth;
 - ▶ that the patient is under the care of the given team;
 - ▶ that the consultant that heads the team is responsible for the patient;
 - ▶ that the patient is on the ward.
- 2 It informs the administrator of the ward to which the patient has been admitted.

B Treat Patient. The administrator identifies the patient and the doctor; the doctor must be in the team that cares for the patient. The system records that the doctor has treated the patient.

C Discharge Patient. The administrator identifies the discharged patient. All information relating to that patient is removed from the system.

D Transfer Patient. The administrator identifies the patient and the new ward. The new ward should be of the appropriate type and have at least one free bed. The system records:

- ▶ the removal of the patient from their current ward;
- ▶ the admission of the patient to the new ward.

E List Ward's Patients. The administrator identifies the ward. For each patient on the ward the system displays the patient's name and age in years.

F List Team's Patients. The administrator identifies the team. For each patient cared for by the team the system displays:

- ▶ the patient's name;
- ▶ the name of the ward that the patient is on.

G List Patient's Treatment. The administrator identifies the patient. The system displays:

- ▶ the name of the consultant responsible for the patient;
- ▶ the team code of the team that cares for the patient;
- ▶ the name of each doctor who has treated the patient and, for each such junior doctor, their grade (1, 2 or 3).

Acceptance tests

A Test that, when there is an appropriate ward available:

- ▶ the admission of a patient is recorded correctly;
- ▶ the administrator is informed of the ward to which the patient has been allocated.

Test that, when no appropriate ward is available, an attempt to record the admission of a patient results in the administrator being informed that there is no appropriate ward available.

B Test that the treatment of a patient, by a doctor on the team that cares for the patient, is recorded correctly.

C Test that the discharge of a patient is recorded correctly.

D Test that the transfer of a patient to an appropriate ward with an empty bed is recorded correctly.

E Test that a list of patients on a ward is produced correctly when there are no patients on the ward.

Test that a list of patients on a ward is produced correctly when there are some patients on the ward.

- F** Test that a list of patients cared for by a team is produced correctly when there are no patients cared for by the team.
Test that a list of patients cared for by a team is produced correctly when there are some patients cared for by the team.
- G** Test that a list of a patient's treatment is produced correctly when no doctors have treated the patient.
Test that a list of a patient's treatment is produced correctly when some doctors have treated the patient.

The Hospital System requirements above do not describe the user interface as a full requirements document would. For example, it would be usual to describe how the administrator should identify patients: by typing in a patient's name, or selecting a name from a list, for example. Such issues will be taken up when you study user interface design and development in *Units 11, 12 and 13*.

In this section, you have applied the skills of elicitation and analysis you have learnt in the unit to the process of developing a requirements document for the Hospital System. It is important to note that this system is not intended to mimic even a cut-down version of a 'real' system; any such system would be far too complex for this introduction to software development. Accordingly we have chosen the above requirements so that they can be used to illustrate the main activities involved in the software development process without introducing unwanted complexities.

This requirements document for the Hospital System is taken in *Unit 3* as the starting point for the development of a prototype software system.

5

The Shopping System

In the previous section you participated in the production of a simplified requirements document for the Hospital System, which focused on the system domain, the use cases and the acceptance tests.

In this section you are asked to assist in producing a similar document for an online shopping system (referred to as the Shopping System).

5.1

A brief description of the Shopping System

The Open Shopping Company sells computing products on its website. At any given time there are usually many items of each product in stock, although occasionally a product may go out of stock.

The company has a number of registered customers who can sign on for a session on the website to browse product details and, if they wish, to open an order. The customer can add items to their order; if a product is out of stock, an alternative may sometimes be suggested. The customer can add as many items to their order as they wish. At any time during the session the customer should be able to see what is in their current order, and be able to discard their order, or submit it if it is not empty.

Staff members have to be able to record new products and amend existing product details. They also need to be able to access details of the products and of registered customers, and to find out which customers currently have sessions.

Exercise 15

Identify some tasks that are to be performed by the users of this system that you may want to consider through the requirements elicitation and analysis process.

Discussion.....

There are many tasks arising out of even the small amount of information given in the description above, and many more may evolve through the requirements elicitation process. Here are just six that arise from the tasks the customer needs to be able to perform during a session.

- 1 List product details.
- 2 Create an order.
- 3 Add an item to an order.
- 4 Display order details.
- 5 Submit an order.
- 6 Discard an order.

Of course these are not the only tasks to be performed – there are also tasks to be performed by the customer before they start a session and tasks to be performed by the staff members.

Exercise 16

Produce a list of some of the questions that you could ask to help you in the requirements elicitation and analysis of the Shopping System. You should not spend a huge amount of time on this very open-ended exercise, so perhaps time yourself and see how many questions you can produce in just fifteen minutes!

Discussion.....

Again there are no right or wrong answers to this exercise, but some of the questions you might have asked are given below. Many other, equally important, questions could also have been asked. Here we have categorised them according to possible tasks and grouped these possible tasks under three different headings.

Customer tasks before a session

- A Record the registration of a customer
 - How does a customer register?
 - What data do they have to provide?
 - Can they do this themselves?
 - What information do you record about customers?
- B Start a session
 - How does a customer start a session?
 - How can you be sure that a customer is who they claim to be; is there any security?

Customer tasks during a session

- C Terminate a session
 - What is recorded about the terminated session?
- D List submitted orders
 - Is there a time limit for displaying submitted orders?
 - If so, how long?
 - What information is displayed about a submitted order?
 - Can a customer see which orders have been dispatched?
- E List product details
 - What information do customers need to see?
- F Create an order
 - Can a customer create more than one order per session?
 - If so how many can they create?
 - If not what happens if they try to create more than one?
- G Add an item to an order
 - What happens if an item is out of stock?
 - Do all products have alternatives?
 - Does the customer have a choice about accepting an alternative?
 - What happens if the alternative is out of stock?
 - What is recorded when an item is added to an order?
 - How do customers order more than one item of a product?
 - Can a customer change their mind about an item and remove it from their order?
 - Is there an upper limit to the amount customers can order?

H Display order details

What information does the customer see?

I Submit an order

What happens if a customer tries to submit an empty order?

What is recorded when an order is submitted?

J Discard an order

What information is kept about a discarded order?

Staff member tasks

K List details of registered customers

What information needs to be displayed about registered customers?

L List details of those customers with a current session

What information needs to be displayed about customers with a current session?

M Record details of a new product

What needs to be recorded about a new product?

Is an alternative product identified for each new product?

N List details of products

What information needs to be displayed when products are listed?

Is the information displayed to the staff and customers the same?

O Record a change to a product

What details of a product may be changed?

When the details of a product are changed are existing orders for items of that product affected by the change?

P List details of submitted orders

Do staff members need to be able to see details of orders customers have submitted?

Exercise 17

As a result of elucidating what information should be displayed about registered customers (K above), the following has been identified as a use case of the Shopping System.

List Registered Customers. For each registered customer the system displays the name, address and customer number.

What acceptance tests are needed for this use case?

Discussion.....

There are two possibilities: either there are no registered customers, or there are registered customers. Therefore two acceptance tests are required, as follows.

- 1 Test that a list of registered customers is displayed correctly if there are no registered customers.
- 2 Test that a list of registered customers is displayed correctly if there are some registered customers.

5.2 The requirements document

After extensive questioning you could arrive at a requirements document such as the following.

The requirements document for the Shopping System

This requirements document is reproduced in Section 6 of the *Handbook*.

System domain

The Open Shopping Company sells computing products on its website. The company has a number of registered customers. A registered customer may visit the website to view product details, and to make and place an order. A customer must be authenticated by the system before such an online session can commence. A customer may only be involved in a single session at any one time.

Once a customer has started an online session, and opts to create an order, an empty order is created, and the customer can add one item of an available product to their order at a time. As many items as required may be added to an order. Multiple items of the same product must be added singly.

If a selected product is out of stock, and the company has a suitable alternative in stock, the system will suggest this alternative to the customer, who may select or reject it. Alternative products are paired, so that if product A is considered by the company to be a suitable alternative to product B, then product B is considered to be a suitable alternative to product A. Each product may only have a single alternative and not all items have alternatives.

The customer may submit or discard an order at any time during the session. It is not possible to add any further items to an order once it has been submitted. A customer may submit more than one order during a single session, but may only have one unsubmitted order at a time.

Use cases

The system will allow a customer to do the following before a session.

- A Record Customer Registration.** The customer provides the system with the customer's name, address and chosen password. The system provides a unique customer number and records the customer's details. The customer number is displayed.
- B Start Session.** The customer provides the system with a customer number and a password. If the entered password matches the password recorded for the customer with that number, the system records a current session for that customer. If the customer is not authenticated in this way, the system displays an appropriate message.

The system will allow a customer with a current session to do the following.

- C Terminate Session.** The system records that the customer no longer has a current session. If the customer has an unsubmitted order this is discarded (see below). All information about the session is removed from the system.
- D List Submitted Orders.** For each order submitted by the customer, the system displays the product code, description and price (at time of ordering) of each item in the order, and the order number, date submitted and total cost of the order.

- E List Products.** For each product the system displays the product code, description and price.
- F Create Order.** The system creates an empty order for the customer and generates a unique order number.
- G Add Item.** The customer identifies a product. If the product is in stock, the system records that an item of the product has been added to the customer's current order. If the product is out of stock, the system displays an appropriate message. If there is an alternative to the product, and there are items of this alternative in stock, then the alternative is suggested to the customer. If the customer accepts the alternative the system records that an item of the alternative product has been added to the customer's current order. The system decrements the stock level of the product added to the order.
- H Display Order.** The system displays the product code, description and price (at time of ordering) of each item in the order, and the date, order number and total cost of the order.
- I Submit Order.** If the order contains at least one item, the system records that the order has been submitted.
- J Discard Order.** The system increments the stock level of each product with items in that order. All information about the order is removed from the system.

The system will allow a staff member to do the following.

- K List Registered Customers.** For each registered customer the system displays the name, address and customer number.
- L List Customers With Current Session.** For each customer with a current session the system displays the time the session started, the customer's name and customer number, and indicates whether that customer has an unsubmitted order.
- M Record New Product.** The staff member provides the system with the product's unique product code, description, stock level and price. If an alternative product is available, the staff member identifies it. Such an alternative product should not already be an alternative for another product. The system records the new product and, if relevant, records that it and the identified alternative are alternatives.
- N List Products.** For each product the system displays the product code, description, price and stock level.
- O Record Product Change.** The staff member identifies a product and either provides the system with a new stock level or price, or identifies an alternative product (which should not already be an alternative for another product). The system records the changed product details for one or both products. Items of this product that have already been added to a customer's order are not affected by an updated price.
- P List Submitted Orders.** The staff member identifies a customer. The system lists that customer's submitted orders as for use case D.

Acceptance tests

- A** Test that a new customer is registered correctly.
- B** Test that a session is initiated correctly for a legitimate customer.
Test that an attempt to initiate a session by a non-legitimate customer results in an appropriate message.
- C** Test that a session is terminated correctly if the customer has no unsubmitted order.
Test that a session is terminated correctly if the customer has an unsubmitted order.

- D** Test that a list of submitted orders is produced correctly when there are no orders submitted by the customer.
Test that a list of submitted orders is produced correctly when there are orders submitted by the customer.
- E** Test that a list of products is produced correctly when there are no products recorded.
Test that a list of products is produced correctly when there are products recorded.
- F** Test that an empty order is produced correctly.
- G** Test that a product item is added correctly to an order if it is available.
Test that a product item is added correctly if the original product item is not available, but an alternative is available and is accepted.
Test that a product item is not added if the original product item is not available and there is no alternative available.
Test that a product item is not added if the original product item is not available, an alternative is available and is not accepted.
- H** Test that an order is displayed correctly when there are no items in the order.
Test that an order is displayed correctly when there are items in the order.
- I** Test that a non-empty order is correctly recorded as submitted.
- J** Test that an order is correctly discarded when there are no items in the order.
Test that an order is correctly discarded when there are items in the order.
- K** Test that a list of registered customers is produced correctly when there are no customers recorded.
Test that a list of registered customers is produced correctly when there are customers recorded.
- L** Test that a list of customers with current sessions is produced correctly when there are no customers with current sessions.
Test that a list of customers with current sessions is produced correctly when there are customers with current sessions, none of whom have an unsubmitted order.
Test that a list of customers with current sessions is produced correctly when there are customers with current sessions, some of whom have an unsubmitted order.
- M** Test that a new product with no identified alternative is recorded correctly.
Test that a new product with an identified alternative is recorded correctly and that the product and the alternative are recorded as alternatives.
- N** Test that a list of products is produced correctly when there are no products recorded.
Test that a list of products is produced correctly when there are products recorded.
- O** Test that, when changing the alternative product is not part of the change, a product change is recorded correctly.
Test that, when changing the alternative product is part of the change, a product change is recorded correctly, including the fact that the new product and the alternative are recorded as alternatives.
- P** Given a customer:
- ▶ test that a list of submitted orders is produced correctly when there are no orders submitted by that customer;
 - ▶ test that a list of submitted orders is produced correctly when there are some orders submitted by that customer.

In this section you have applied your elicitation and analysis skills to the process of developing a requirements document for the Shopping System. As with the Hospital System, it is important to appreciate that the requirements for the Shopping System are not intended to be entirely realistic: they have been chosen as a suitable vehicle for your learning in this area. In a full requirements document for such a system you would expect issues such as the user interface, security and the use of existing databases to be discussed.

In *Unit 4* you will take this requirements document for the Shopping System as the starting point for developing a prototype software system.

6

Summary

In this unit you have learnt that the first step in developing a new software system is to elicit and analyse the requirements of the proposed system. In order to develop an understanding of the requirements, a specialised software developer, called an analyst, gathers information from the client organisation through a variety of questioning techniques.

The analyst then analyses the requirements and specifies them in a requirements document that forms the basis of a contract between the client and the software developers and which contains a clear description of:

- 1 the system domain;
- 2 the behavioural requirements (i.e. tasks that the system will carry out) in the form of use cases;
- 3 the acceptance tests that will be performed in order to check that the system provides all the required behaviour.

The requirements document will also contain other information, such as the constraints on the system and project management information. However, this course will focus on the three elements listed above.

Finally, you applied what you learnt about elicitation and analysis to the creation of requirements documents for the Hospital System and the Shopping System, which will be used and developed in later units of this course.

LEARNING OUTCOMES

After studying this unit you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ explain why requirements specification is an important phase in the software development process;
- ▶ describe some of the processes by which requirements can be elicited and analysed;
- ▶ examine an initial description of a software system and suggest questions to elicit further information about the requirements of the system;
- ▶ describe the main elements of a requirements document and in particular what is meant by system domain, the behavioural requirements (use cases) and acceptance tests;
- ▶ identify and specify appropriate tasks and use cases from an analysis of the elicited requirements;
- ▶ discuss the difference between behavioural requirements and constraints, and identify examples of each;
- ▶ develop acceptance tests for given use cases.

Glossary

acceptance tests A series of tests devised to demonstrate that a completed system can meet all the behavioural requirements set out as use cases in the requirements document.

acceptance testing Testing which aims to convince the client that the system works as specified.

analyst A generic term for a person who takes on the role (or a combination of the roles) of a business analyst, systems analyst or requirements analyst.

behavioural requirement A specific task that the system being developed must be able to do.

business analyst A person who understands the business systems within an organisation.

business area The real-world context within which the system is required to operate.

business system A business system is a combination of people and software working together to meet a particular set of business objectives.

constraint (on the system) A limitation on the system or its development.

requirements analysis The process of analysing and distilling the information from requirements elicitation into a clear and coherent description of the system and specification of its behaviour.

requirements analyst A person who is an expert in eliciting and analysing the requirements of a new software system.

requirements creep A situation where the software developers come under pressure to deliver in excess of what was originally agreed.

requirements document The specification of the requirements of the system produced after negotiation between the client and the analyst.

requirements elicitation The act of finding out the requirements of a system by consulting with the stakeholders.

scope The breadth of services that the system is expected to provide.

stakeholder Anyone who has an interest in a system including the client, the future users, analysts and developers.

system domain The real-world context specific to the software system being developed.

use case A concise description of a specific task that a user of the system must be able to perform.

References

Cusumano, M.A. and Yoffie, D. (1999) 'Software Development on Internet Time', *Computer*, vol. 32, no. 10, pp. 60–69.

Hay, D.C. (2002) *Requirements Analysis*, Prentice Hall PTR.

Kulak, D. and Guiney, E. (2003) *Use Cases: Requirements in Context*, Addison-Wesley Professional.

Index

- A
 - acceptance test 18, 39
 - acceptance testing 19
 - analyst 8–9, 39
- B
 - behavioural requirement 12–13, 18
- business
 - area 7
 - context (of a system) 18
 - language 11
 - system 7
- business analyst 7
- C
 - client 9
 - code 9
 - constraint 20, 39
- D
 - designer 9
- E
 - eXtreme Programming (XP) 11
- H
 - Hospital System 5, 24, 39
- J
 - Joint Application Development (JAD) 11
- M
 - model 9
- P
 - programmer 9
 - project management 18
 - prototype 11
- R
 - Rapid Application Development (RAD) 11
 - requirements
 - analysis 13
 - creep 23
 - document 7, 17, 32, 35
 - elicitation 9
 - specification 5, 7
 - requirements analyst 7
- S
 - scope 12
 - sequence diagram 9
 - Shopping System 5, 32
 - stakeholder 9, 11
 - analyst 9
 - client 9
 - designer 9
 - programmer 9
 - user 9
 - system domain 7, 18, 39
 - systems analyst 7
- T
 - techniques (for eliciting requirements) 11
- U
 - use case 13, 18–19, 39
 - user 9

